

Wissenschaftlich angeleitete Berufspraxis (WAB)

Performance Evaluation of Classical and Post-Quantum Digital Signature Algorithms

For the course

“Algorithmen und Datenstrukturen“

In the degree program BIN-T25-F2

submitted to the Department of

Informatik

of

Provadis School of International Management and Technology

from

Luca Grupinski

Student ID E198

BIN-T25-F2

luca-paul.grupinski@stud-provadis-hochschule.de

First Reviewer:

Prof. Dr. Jörg Daubert

End of the processing period:

30.09.2026

Table of contents

Glossary (alphabetically sorted)	ii
List of tables	v
List of Abbreviations	vi
1 Introduction	1
1.1 Problem Statement and Relevance.....	1
1.2 Research Question and Objective	2
1.3 Scope of the Study	2
2 Theoretical Background	3
2.1 Fundamentals of Digital Signatures.....	3
2.2 Classical Digital Signatures: RSA and ECDSA	3
2.3 Post-Quantum Cryptography and ML-DSA	4
2.4 Relevant Performance Metrics.....	4
3 Research Methodology	5
3.1 Experimental Setup	5
3.2 Benchmark Algorithm	6
3.3 Data Collection and Evaluation Criteria	6
4 Results	7
4.1 Key Generation Performance.....	7
4.2 Signature Generation and Verification	7
4.3 Memory Consumption	8
4.4 Key and Signature Sizes	8
5 Discussion	9
5.1 Comparison with Existing Literature.....	9
5.2 Implications for Practical Deployment	10
6 Conclusion	11
6.1 Answer to the Research Question	11
6.2 Outlook	11
Bibliography	vii
AI Directory	viii
Declaration of Originality	ix

Glossary (alphabetically sorted)

Term	Definition / Explanation
Certificate	Data structure that binds a public key to an identity, such as a person or a server, and is itself protected by the digital signature of a certification authority.
CRYSTALS-Dilithium	Lattice-based digital signature scheme (CRYSTALS: Cryptographic Suite for Algebraic Lattices) submitted to the NIST post-quantum standardization process; standardized with minor changes as ML-DSA.
DER encoding	Distinguished Encoding Rules: standardized binary format for structured data, commonly used to store and exchange keys and signatures, for example in certificates.
Digital signature	Cryptographic mechanism consisting of key generation, signature generation and signature verification. The signer signs with a private key, and anyone can verify the signature with the corresponding public key. It ensures integrity, authenticity and non-repudiation.
ECDSA	Elliptic Curve Digital Signature Algorithm: elliptic-curve variant of the Digital Signature Algorithm (DSA) whose security relies on the discrete logarithm problem on elliptic curves. ECDSA-P256 uses the NIST curve P-256 and offers 128-bit security.
FIPS 204	Federal Information Processing Standard published by NIST on 13 August 2024 that specifies ML-DSA as a standardized post-quantum digital signature scheme.
Hash value	Short fingerprint of fixed length that a hash function, such as SHA-256, computes from a message of arbitrary length. In practice, the hash value is signed instead of the message itself.
Key pair	Combination of a private key, which is kept secret and used for signing, and a public key, which may be published and is used for verification.

ML-DSA	Module-Lattice-Based Digital Signature Algorithm: post-quantum signature scheme standardized in FIPS 204 and derived from CRYSTALS-Dilithium. It is defined in the parameter sets ML-DSA-44, ML-DSA-65 and ML-DSA-87, which differ in security category and in key and signature sizes.
Module Learning With Errors (MLWE)	Lattice problem on which the security of ML-DSA is based: recovering a secret from a system of linear equations over a polynomial ring that has been disturbed by small random errors. It is considered hard even for quantum computers.
NIST security category	Classification of the strength of post-quantum algorithms defined by NIST. A scheme in category 3, such as ML-DSA-65, is claimed to be at least as hard to break as a block cipher with a 192-bit key, such as AES-192.
Non-repudiation	Security property ensuring that a signer cannot later deny having created a signature, because only the holder of the private key can produce it.
Peak working set	Maximum amount of physical memory (RAM) that a process occupies at any time during its execution, as reported by Windows; used in this paper to measure memory consumption.
Post-quantum cryptography (PQC)	Public-key algorithms for which no efficient quantum attacks are known, based for example on lattice, code or hash problems. Unlike quantum cryptography, they run on conventional hardware.
Quantum computer, cryptographically relevant	Quantum computer that is large and reliable enough to break currently used public-key schemes such as RSA and ECDSA in practice.
Rejection sampling	Technique used in ML-DSA signing: candidate signatures that could leak information about the private key are discarded and recomputed until a valid signature is found. The number of iterations therefore varies from signature to signature.

Resource-constrained platform	Device with limited computing power, memory, bandwidth or energy, such as microcontrollers (e.g. the ESP8266), smart cards or IoT devices.
RSA	Public-key scheme published in 1978 by Rivest, Shamir and Adleman, usable for encryption and digital signatures. Its security relies on the difficulty of factoring the product of two large prime numbers. RSA-3072 uses a 3,072-bit modulus and offers 128-bit security.
RSA-PSS	Probabilistic Signature Scheme: standardized padding method for RSA signatures that encodes the hashed message together with a random value before signing, which prevents forgeries that are possible with unpadded RSA.
Security level	Measure of cryptographic strength in bits: an algorithm offers n-bit security if the best known attack requires about as much effort as trying all keys of an n-bit symmetric cipher.
Seed	Short random value from which further values are derived deterministically. In ML-DSA, a 32-byte seed is sufficient to reconstruct the complete private key of 4,032 bytes.
Shor's algorithm	Quantum algorithm published by Peter Shor in 1994 that efficiently solves the integer factorization and discrete logarithm problems and would therefore break RSA and ECDSA on a sufficiently large quantum computer.
Stack	Memory region in which a program stores local variables and function call data. On microcontrollers, the required stack size is a key measure of an algorithm's RAM demand.
Warm-up run	Execution of an operation before the actual measurement whose result is discarded in order to exclude distortions caused by loading and initialization effects.

List of tables

Table 1: Key parameters of the investigated methods according to literature and specifications	5
Table 2: Measured execution times in μs (per 1,000 measurements).....	7
Table 3: Increase in peak memory in KiB, median (maximum) over 30 processes	8
Table 4: Measured key and signature sizes in bytes	9

List of Abbreviations

AES	Advanced Encryption Standard
Cf.	Confer (compare)
CRYSTALS	Cryptographic Suite for Algebraic Lattices
DDR5	Double Data Rate 5
DER	Distinguished Encoding Rules
DSA	Digital Signature Algorithm
ECDSA	Elliptic Curve Digital Signature Algorithm
FIPS	Federal Information Processing Standard
FN-DSA	FFT over NTRU-Lattice-Based Digital Signature Algorithm
IoT	Internet of Things
KiB	Kibibyte (1,024 bytes)
MiB	Mebibyte (1,024 KiB)
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
MLWE	Module Learning With Errors
NIST	National Institute of Standards and Technology
p. / pp.	Page / pages
PQC	Post-Quantum Cryptography
PSS	Probabilistic Signature Scheme
RAM	Random Access Memory
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
SLH-DSA	Stateless Hash-Based Digital Signature Algorithm
WAB	Wissenschaftlich angeleitete Berufspraxis

1 Introduction

1.1 Problem Statement and Relevance

Digital signatures ensure integrity, authenticity, and non-repudiation; they are used for digital certificates, legally binding contract signings, and secure software updates, among other things.¹ The widely used RSA and ECDSA methods are based on the factorization problem and the discrete logarithm problem on elliptic curves, respectively, which could be efficiently solved by a sufficiently powerful quantum computer using the Shor's algorithm.² According to NIST, the security of such signature methods is at risk if large-scale quantum computers are realized;³ some experts believe that a device that can break today's encryption methods is possible within a decade.⁴

In response, NIST initiated a standardization process for post-quantum cryptography (PQC) in 2016, which resulted in the signature method ML-DSA, derived from CRYSTALS-Dilithium and based on the Module Learning With Errors (MLWE) problem.⁵ ML-DSA is also considered secure against attackers with a large-scale quantum computer and has been the primary standard for digital signatures in FIPS 204 since August 2024; at the same time, NIST encourages the transition to be initiated as early as possible.⁶

In addition to security, however, efficiency is crucial for migration: Public key algorithms are computationally intensive, and their runtime can become a limiting factor on devices with limited resources, such as smart cards or IoT devices.⁷ Key and signature sizes for embedded systems and network protocols with limited bandwidth are just as critical.⁸ This is a key challenge, because the keys and signatures of PQC methods such as ML-DSA are significantly larger than those of ECDSA.⁹ Previous studies also show that ML-DSA achieves comparable or sometimes better runtimes than ECDSA, but requires more memory, and that PQC algorithms do not necessarily meet all system requirements in resource-constrained environments.¹⁰

¹ Cf. Paar/Pelzl (2010), pp. 259, 263–264

² Cf. Tovma/Balagura (2025), p. 112; Schemitt et al. (2025), p. 1

³ Cf. NIST (2024a), p. 1

⁴ Cf. NIST (2024b)

⁵ Cf. NIST (2024a), p. 1

⁶ Cf. NIST (2024a), p. 1; NIST (2024b)

⁷ Cf. Paar/Pelzl (2010), pp. 156–157; Dziechciarz/Niemiec (2025), p. 7

⁸ Cf. Tovma/Balagura (2025), p. 112

⁹ Cf. Schemitt et al. (2025), p. 6

¹⁰ Cf. Schemitt et al. (2025), p. 2

The comparative studies under consideration only partially cover these requirements. Tovma and Balagura compare RSA, ECDSA and CRYSTALS-Dilithium, but measure on a MacBook Air M2 with 16 GB of RAM and without recording memory consumption.¹¹ Dziechciarz and Niemiec compare PQC methods on a general-purpose computer exclusively with RSA-2048 and name the comparison with elliptic curves as a task for future research.¹² Schemitt et al. measure on laptops and a desktop computer, use ECDSA exclusively as a classic reference, do not collect key and signature sizes, and defer studies on IoT devices to future work.¹³ A joint comparison of all three schemes in terms of runtime, memory consumption and key and signature sizes on a uniform implementation basis is therefore missing in the studies under consideration.

1.2 Research Question and Objective

This leads to the following research question: How do RSA-3072, ECDSA-P256 and ML-DSA-65 compare in terms of computational performance, memory consumption and key and signature sizes, and what are the implications for their practical deployment?

The aim is to compare the three schemes on the basis of own measurements of key generation, signature generation and verification, memory consumption and key and signature sizes, to compare the measured values with published studies and official specifications, and to derive implications for practical deployment. This paper tests the hypothesis that ML-DSA-65 requires larger keys, larger signatures and more memory than RSA-3072 and ECDSA-P256, but achieves computational performance in the same order of magnitude as the classical schemes.

1.3 Scope of the Study

The work is limited to RSA-3072 and ECDSA-P256 as common classical reference methods as well as ML-DSA-65 as NIST-standardized post-quantum signature methods. The classical parameters correspond to a security level of 128 bits,¹⁴ while ML-DSA-65 claims NIST security category 3;¹⁵ this difference is taken into account when interpreting the results. Other PQC signature methods such as SLH-DSA and FN-DSA (Falcon),¹⁶ methods for encryption and key

¹¹ Cf. Tovma/Balagura (2025), pp. 112–113

¹² Cf. Dziechciarz/Niemiec (2025), pp. 14, 16

¹³ Cf. Schemitt et al. (2025), pp. 2, 6

¹⁴ Cf. Paar/Pelzl (2010), p. 156

¹⁵ Cf. NIST (2024a), p. 15; Schemitt et al. (2025), p. 3

¹⁶ Cf. Schemitt et al. (2025), p. 1

exchange as well as an independent security analysis are not the subject of the work. Methodologically, a literature analysis is combined with own benchmarks, which are carried out with an established cryptographic library on a Windows computer (see Section 3.1).

2 Theoretical Background

2.1 Fundamentals of Digital Signatures

Digital signatures are public key algorithms: the signer signs with his private key, the recipient verifies the signature with the corresponding public key. Since only the owner of the private key can generate valid signatures, a signed message can be clearly assigned to its originator. Digital signatures thus guarantee integrity, authenticity and non-repudiation, but not confidentiality.¹⁷ Each signature procedure includes key generation, signature generation and signature verification; in practice, it is usually not the message itself that is signed, but its hash value.¹⁸

2.2 Classical Digital Signatures: RSA and ECDSA

RSA was introduced in 1978 by Rivest, Shamir and Adleman. Its security is based on the fact that the product of two large prime numbers is practically impossible to factor. In practice, RSA is used with a padding method such as RSA-PSS. For performance, it is crucial that key generation requires the complex generation of large prime numbers, verification is very fast thanks to a short public exponent, and signature generation remains comparatively slow. In addition, an RSA signature is as long as the module, i.e. 3,072 bits in the case of RSA-3072.¹⁹

ECDSA is the elliptic curve-based counterpart to the Digital Signature Algorithm (DSA) and is based on the discrete logarithm problem on elliptic curves. Because smaller parameters are sufficient, it enables faster calculations and smaller keys – an advantage especially with limited computational performance, storage capacity, bandwidth or energy.²⁰ For the same security level of 128 bits, ECDSA requires 256 bits, while RSA requires 3,072 bits.²¹ The P-256 curve is one of the standard curves recommended by NIST.²²

¹⁷ Cf. Paar/Pelzl (2010), pp. 262–264

¹⁸ Cf. Paar/Pelzl (2010), pp. 263, 269

¹⁹ Cf. Paar/Pelzl (2010), pp. 176, 183, 267–269

²⁰ Cf. Johnson/Menezes (2000), p. 3

²¹ Cf. Paar/Pelzl (2010), pp. 156, 183

²² Cf. Johnson/Menezes (2000), pp. 36, 38

2.3 Post-Quantum Cryptography and ML-DSA

Post-quantum cryptography includes public-key methods against which no quantum attacks are known, such as lattice, code, or hash-based methods; RSA and ECDSA, on the other hand, are vulnerable to quantum computers.²³ PQC methods run on conventional hardware; in the NIST standardization process, they were evaluated not only for security, but also for performance as well as key and signature size.²⁴

The ML-DSA, standardized in FIPS 204 in 2024, is derived from CRYSTALS-Dilithium and is based on lattice problems, specifically the module learning with errors problem; it is also considered secure against attackers with large quantum computers.²⁵ During signing, candidates are discarded until a valid signature is created, which is why the number of runs varies; verification does not require such a loop.²⁶ ML-DSA-65 is assigned to NIST security category 3 (see Table 1). Implementations are allowed to pre-store intermediate values for faster signing and verification, or to store the private key as a short seed to save space.²⁷

2.4 Relevant Performance Metrics

The methods result in three key figures for comparison. The first is the execution time of key generation, signature generation and verification. It determines practical usability and is averaged over many repetitions in comparable studies.²⁸ The extent to which the individual operations are weighted depends on the use; in public key infrastructures, for example, certificates are signed once, but verified many times.²⁹ The second key figure is key and signature sizes, which are critical for embedded systems and networks with limited bandwidth.³⁰ The third is the memory consumption during execution; ML-DSA is reported to have a higher demand here than for ECDSA.³¹ Table 1 summarizes the most important parameters from the literature and specification.

²³ Cf. Paar/Pelzl (2010), pp. 168–169

²⁴ Cf. Dziechciarz/Niemiec (2025), pp. 2, 4

²⁵ Cf. NIST (2024a), p. 1

²⁶ Cf. NIST (2024a), pp. 15, 23, 26–27

²⁷ Cf. NIST (2024a), pp. 15–16

²⁸ Cf. Dziechciarz/Niemiec (2025), pp. 5–7; Schemitt et al. (2025), pp. 3–4

²⁹ Cf. Paar/Pelzl (2010), p. 267

³⁰ Cf. Tovma/Balagura (2025), p. 112

³¹ Cf. Schemitt et al. (2025), p. 2

Table 1: Key parameters of the investigated methods according to literature and specifications

	RSA-3072	ECDSA-P256	ML-DSA-65
Security assumption	Factorization	Discrete logarithm on elliptic curves	Lattice problem (MLWE)
Security level	128-bit	128-bit	NIST Category 3
Quantum-resistant	no	no	yes (according to current knowledge)
Public Key	384 Byte (modulus)	65 bytes (uncompressed point)	1,952 Byte
Signature Length	384 Byte	64 Byte	3,309 Byte

Source: Own illustration according to Paar/Pelzl (2010), pp. 156, 169, 267, 283; Johnson/Menezes (2000), pp. 3, 24, 38; NIST (2024a), pp. i, 1, 15–16. The values for RSA and ECDSA are converted from the bit lengths, without a encoding overhead

3 Research Methodology

3.1 Experimental Setup

An ESP8266 microcontroller (ESP-12F module) was originally planned as a resource-limited measurement platform. However, this plan proved infeasible within the scope of this paper, because the RAM of the ESP8266 is not sufficient for the speed-optimized ML-DSA-65 implementations considered. According to the manufacturer, only about 50 kB of heap and data are available with active Wi-Fi.³² For Dilithium3, which corresponds to the ML-DSA-65 parameter set,³³ optimized implementations for the more powerful Cortex-M4 require around 69 to 74 kB of stack for signature generation and around 58 kB for verification.³⁴ Both operations thus significantly exceed the available memory. Without ML-DSA-65, the intended comparison of the three methods would not have been possible, so the measurements are instead carried out on a Windows computer.

This is equipped with an AMD Ryzen 9 7900X (12 cores, 24 logical processors, base clock 4.7 GHz) and 32 GB DDR5 RAM. All three methods are implemented in Python 3.13 with the cryptography library in version 50.0.1, which – as with Schemitt et al. for ECDSA – serves as an interface to a C implementation.³⁵ Since it uses the same backend OpenSSL 4.0.2 for all methods, the measurements are based on a uniform implementation basis. RSA-3072 with the public exponent 65,537 and the padding method RSA-PSS, ECDSA with the curve P-256 and

³² Cf. Espressif Systems (2025), p. 8

³³ Cf. NIST (2024a), p. 1; Schemitt et al. (2025), pp. 3–4

³⁴ Cf. Abdulrahman et al. (2022), p. 18

³⁵ Cf. Schemitt et al. (2025), p. 4

SHA-256 as well as ML-DSA-65 according to FIPS 204 are examined.³⁶ In order to reduce interference, the measurement process is pinned to a single logical processor core and executed with high priority; Dziechciarz and Niemiec also limited their measurement environment to one processor.³⁷ No other applications are running during the measurements.

3.2 Benchmark Algorithm

For each method, key generation, signature generation and signature verification are measured separately. Each signature is formed via a new, randomly generated message of 1 KB, as messages up to 1 KB are the most common use case.³⁸ Before each series of measurements, 100 warm-up runs are performed, which are discarded because initial executions can be distorted by loading and initialization effects.³⁹ Each operation is then executed 1,000 times and its runtime is measured with the high-resolution Python function `time.perf_counter_ns()`; Python's automatic garbage collection is deactivated. Each generated signature is verified so that faulty runs are detected.

The `psutil` library records the memory consumption as the peak working set of the process. Measurements are taken in freshly started processes: After loading the library and the required keys, an initial value is determined, then the operation is executed ten times and the increase in peak RAM is recorded. This process is repeated in 30 processes per operation. The key and signature sizes are determined by the byte sequences that the library outputs, in raw format and in the usual DER encoding.⁴⁰

3.3 Data Collection and Evaluation Criteria

All individual values are stored as raw data and condensed into mean, median, and standard deviation for each method and operation;⁴¹ for the memory, the median and maximum are reported across the 30 processes. The runtime per operation, the memory growth, and the key and signature sizes are evaluated, each in absolute terms and in relation to RSA-3072 and ECDSA-P256. The measured quantities are compared with the specification values from Table 1, and the runtimes are compared with the results of Tovma and Balagura, Dziechciarz and Niemiec, and Schemitt et al. According to the hypothesis, ML-DSA-65 is considered practical if its

³⁶ Cf. Paar/Pelzl (2010), pp. 267–269, 286–287

³⁷ Cf. Dziechciarz/Niemiec (2025), p. 6

³⁸ Cf. Dziechciarz/Niemiec (2025), pp. 5, 16

³⁹ Cf. Dziechciarz/Niemiec (2025), p. 6; Schemitt et al. (2025), p. 4

⁴⁰ https://github.com/lucagrp06/LucaGrupinski_WAB_PQC-Signaturen_Performance

⁴¹ Cf. Schemitt et al. (2025), p. 3

runtimes are in the same order of magnitude as those of the classical schemes; its memory and size requirements are assessed against the resources of the respective target platform.

4 Results

All 3,000 signatures generated were successfully verified.

4.1 Key Generation Performance

The methods differ most strongly in key generation (Table 2). ECDSA-P256 generates a key pair in a median time of 10.0 μ s, ML-DSA-65 in 121.9 μ s. RSA-3072, on the other hand, requires a median of 94.1 ms and is thus about 770 times slower than ML-DSA-65 and about 9,400 times slower than ECDSA-P256. In addition, the RSA key generation is highly scattered (standard deviation 65.3 ms), which corresponds to the randomly dependent prime number search (see Section 2.2).

4.2 Signature Generation and Verification

When signing, ECDSA-P256 is the fastest with a median of 16.1 μ s. RSA-3072 requires 474.1 μ s, ML-DSA-65 537.4 μ s; ML-DSA-65 is thus about 13 % slower than RSA-3072 and about 33 times slower than ECDSA-P256. The high dispersion of ML-DSA-65 is striking: individual signatures lasted between 225.8 μ s and 5,255.1 μ s, and the mean value of 688.6 μ s is significantly higher than the median. This corresponds to the rejection loop, the number of passes of which varies from signature to signature (see Section 2.3).

In verification, RSA-3072 is the fastest with 30.4 μ s, followed by ECDSA-P256 with 42.4 μ s. ML-DSA-65 requires 121.0 μ s, which is about four times that of RSA-3072 and almost three times that of ECDSA-P256, but hardly scatters. In terms of the median, all operations except RSA key generation take less than one millisecond.

Table 2: Measured execution times in μ s (per 1,000 measurements)

Algorithm	Operation	Median	Mean	Standard deviation	Min	Max
RSA-3072	Key generation	94,127.2	106,805.6	65,254.6	12,814.8	422,854.1
RSA-3072	Signature generation	474.1	523.6	107.6	457.9	1,175.6
RSA-3072	Verification	30.4	30.6	1.1	30.0	53.7
ECDSA-P256	Key generation	10.0	10.0	0.4	9.8	14.7

ECDSA-P256	Signature generation	16.1	16.3	1.0	15.9	39.6
ECDSA-P256	Verification	42.4	42.6	1.7	41.9	78.1
ML-DSA-65	Key generation	121.9	123.5	19.0	120.5	643.3
ML-DSA-65	Signature generation	537.4	688.6	534.4	225.8	5,255.1
ML-DSA-65	Verification	121.0	121.9	3.9	120.6	173.1

Source: Own measurement

4.3 Memory Consumption

Table 3 shows the increase in peak memory per operation. The medians for all methods are between 248 KiB and about 1.7 MiB; the maximum values exceed the median by no more than 14 %. ML-DSA-65 requires the most memory for key generation, but the least for verification. In verification, ML-DSA-65 manages with the smallest increase with 248 KiB, while RSA-3072 requires 668 KiB and ECDSA-P256 1,088 KiB. This does not show a consistently higher memory requirement of ML-DSA-65. It should be noted that the measured variable also includes the initialization of the library (see Section 3.2).

Table 3: Increase in peak memory in KiB, median (maximum) over 30 processes

Procedure	Key generation	Signature generation	Verification
RSA-3072	1,204 (1,324)	600 (600)	668 (760)
ECDSA-P256	1,364 (1,464)	1,672 (1,696)	1,088 (1,152)
ML-DSA-65	1,704 (1,752)	996 (1,000)	248 (248)

Source: Own measurement

4.4 Key and Signature Sizes

The measured sizes of ML-DSA-65 are in line with the specification: the public key is 1,952 bytes, the signature is 3,309 bytes (Table 4). The library stores the private key as a 32-byte seed value, from which the expanded key specified in FIPS 204 as 4,032 bytes can be derived; the specification expressly allows this form.⁴² The signature of ML-DSA-65 is thus about 8.6 times as large as that of RSA-3072 with 384 bytes and about 52 times as large as that of ECDSA-P256 with 64 bytes in raw format or 69 to 72 bytes in DER encoding. The public key is about five times the size of RSA-3072 and about 30 times the size of ECDSA-P256. If the public key and signature are transmitted together, for example in certificates, ML-DSA-65 has 5,261 bytes,

⁴² Cf. NIST (2024a), p. 16

RSA-3072 768 bytes and ECDSA-P256 129 bytes. The values for RSA-3072 and ECDSA-P256 correspond to the sizes derived from the bit lengths in Table 1.

Table 4: Measured key and signature sizes in bytes

Element	RSA-3072	ECDSA-P256	ML-DSA-65	ML-DSA-65 according to FIPS 204
Public key, raw	384	65	1,952	1,952
Public key, DER	422	91	1,974	–
Private key	1,792 (DER)	32 (raw), 138 (DER)	32 (seed), 54 (DER)	4,032 (expanded)
Signature	384	64 (raw), 69–72 (DER)	3,309	3,309

Source: Own measurement; column FIPS 204 according to NIST (2024a), p. 16

5 Discussion

5.1 Comparison with Existing Literature

The measured ranking of the methods largely coincides with comparable studies. As with Tovma and Balagura, the key generation of RSA-3072 is by far the slowest operation, ECDSA-P256 signs the fastest and verifies slower than RSA-3072. The almost identical signing speed of RSA-3072 and Dilithium-3 is also found there (7.54 ms vs. 7.53 ms); in our own measurement, ML-DSA-65 is about 13 % above RSA-3072.⁴³ However, the absolute values of Tovma and Balagura are sometimes more than an order of magnitude higher, about 7.54 ms compared to 474.1 μ s for RSA signature. Since the Bouncy Castle library was measured in Java on a MacBook Air M2, this is probably mainly due to the implementation and platform.⁴⁴

Dziechciarz and Niemiec also observe that RSA is significantly slower in key generation and faster in verification than the post-quantum methods. A slower RSA signature generation, as reported there, is not evident in the present measurements; however, the authors compare RSA-2048 from OpenSSL with post-quantum methods from the liboqs library.⁴⁵

Schemitt et al. find that ML-DSA outperforms ECDSA in most operations, with the exception of key generation and signature at the lowest security level.⁴⁶ In the present measurements, however, ECDSA-P256 is faster than ML-DSA-65 in all three operations. This discrepancy is

⁴³ Cf. Tovma/Balagura (2025), pp. 112–113

⁴⁴ Cf. Tovma/Balagura (2025), pp. 112–113

⁴⁵ Cf. Dziechciarz/Niemiec (2025), pp. 5, 14

⁴⁶ Cf. Schemitt et al. (2025), p. 4

partly explained because Schemitt et al. compare ML-DSA-65 with ECDSA using the curve P-384 and assign P-256 to the lowest level, where ECDSA is also faster in key generation and signature. In addition, ECDSA and ML-DSA come from different libraries there, while here both run on the same OpenSSL backend.⁴⁷

All three studies agree that the switch to ML-DSA or dilithium does not cause a disproportionate computational effort.⁴⁸ This is confirmed by our own measurements (see Section 4.2). The measured sizes also confirm that post-quantum keys and signatures are significantly larger than in ECDSA.⁴⁹ However, the higher memory requirement of ML-DSA reported in the literature is not confirmed;⁵⁰ this is discussed in Section 5.2.

5.2 Implications for Practical Deployment

Three conclusions follow from the results for practical deployment. First, the computational time of ML-DSA-65 is not a fundamental obstacle: All operations have a median of less than one millisecond and thus in the same order of magnitude as RSA-3072. However, the strong dispersion during signing, whose individual values of up to 5.3 ms reach about ten times the median, must be considered; time-critical applications must plan for reserves. The most complex operation remains RSA key generation. Since public key algorithms can generally become a bottleneck on devices with small processors, it is particularly critical there.⁵¹

Second, the bottleneck shifts from computing time to storage and transmission. Public key and signature of ML-DSA-65 together are about 41 times larger than those of ECDSA-P256 (see Section 4.4). For embedded systems and protocols with limited bandwidth, this is a major disadvantage, so security requirements must be weighed against the available storage capacity.⁵²

Third, the RAM decides on which devices ML-DSA-65 can be run at all. Although the memory measurement in this study does not show a higher demand than for the classic methods, it mainly captures the initialization of a desktop library and is therefore not very meaningful for microcontrollers. The case of the ESP8266 is more meaningful: Its available memory is not

⁴⁷ Cf. Schemitt et al. (2025), pp. 3–4

⁴⁸ Cf. Tovma/Balagura (2025), p. 113; Dziechciarz/Niemiec (2025), p. 1; Schemitt et al. (2025), p. 2

⁴⁹ Cf. Schemitt et al. (2025), p. 6

⁵⁰ Cf. Schemitt et al. (2025), p. 2

⁵¹ Cf. Paar/Pelzl (2010), p. 157

⁵² Cf. Tovma/Balagura (2025), p. 112; Schemitt et al. (2025), p. 6

sufficient for ML-DSA-65 (see Section 3.1). On microcontrollers of the Cortex-M4 class with 192 KiB of RAM, on the other hand, the parameter set can be executed.⁵³

6 Conclusion

6.1 Answer to the Research Question

In terms of computational performance, ML-DSA-65 is in the same order of magnitude as RSA-3072: Signature generation requires a median of 537.4 μ s compared to 474.1 μ s, verification 121.0 μ s compared to 30.4 μ s. ECDSA-P256 is the fastest for key generation and signature, RSA-3072 for verification. RSA key generation is by far the most complex operation. In terms of memory consumption, the measurement on the Windows computer does not show a consistently higher demand for ML-DSA-65; the actual limit only becomes visible on microcontrollers, where the RAM of the ESP8266 is not sufficient for the implementations considered. ML-DSA-65 is at a clear disadvantage in terms of sizes: Public key and signature together comprise 5,261 bytes, which is about 41 times as much as ECDSA-P256 and almost seven times as much as RSA-3072.

For practical use, this means that the bottleneck when switching to ML-DSA-65 shifts from computing time to RAM and transmission volume. ML-DSA-65 can therefore be used in most application scenarios; only on severely constrained devices such as the ESP8266 do the implementations considered exceed the available memory.

The hypothesis is thus largely confirmed. ML-DSA-65 requires larger public keys and signatures, while the private key can be stored compactly as a 32-byte seed, and achieves computational performance in the same order of magnitude as the classical schemes. It must be qualified in two points: a higher memory requirement could not be demonstrated in the own measurement, although on microcontrollers the literature values indicate a demand exceeding the memory of the ESP8266. In addition, on resource-constrained platforms, practicality applies only to devices with sufficient memory.

6.2 Outlook

To directly test portability to resource-constrained devices, the measurement should be repeated on a microcontroller with sufficient memory, such as the Cortex-M4 class mentioned in Section 5.2, and the memory requirements there should be recorded at the stack level. In addition, it

⁵³ Cf. Abdulrahman et al. (2022), pp. 15, 18

would be worthwhile to include other signature methods selected by NIST, such as SLH-DSA and Falcon, to investigate integration with protocols such as TLS, VPN, and SSH, and to measure energy consumption on embedded devices.⁵⁴ Since NIST recommends starting the transition to the new standards as early as possible,⁵⁵ migration planning should focus less on computing time and more on the memory and bandwidth reserves of the target platforms.

⁵⁴ Cf. Tovma/Balagura (2025), p. 113

⁵⁵ Cf. NIST (2024b)

Bibliography

- Abdulrahman, A., Hwang, V., Kannwischer, M. J., & Sprenkels, A. (2022). *Faster Kyber and Dilithium on the Cortex-M4*. Springer.
- Dziechciarz, D., & Niemiec, Marcin. (2025). Efficiency Analysis of NIST-Standardized Post-Quantum Cryptographic Algorithms for Digital Signatures in Various Environments. *Electronics*.
- Espressif Systems. (2025). *ESP8266EX Datasheet*. Espressif Systems.
- https://github.com/lucagrp06/LucaGrupinski_WAB_PQC-Signaturen_Performance. (n.d.).
- Johnson, D., & Menezes, Alfred. (2000). *The Elliptic Curve Digital Signature Algorithm (ECDSA)*. Waterloo, Ontario, Kanada: University of Waterloo.
- National Institute of Standards and Technology. (2024). *Module-Lattice-Based Digital Signature Standard*. Gaithersburg, Maryland, USA: National Institute of Standards and Technology.
- National Institute of Standards and Technology. (2024). *NIST*. Retrieved from NIST Releases First 3 Finalized Post-Quantum Encryption Standards: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>
- Paar, C., & Pelzl, Jan. (2010). *Understanding Cryptography*. Berlin, Heidelberg: Springer.
- Rivest, R. L., Shamir, Adi, & Adleman, Leonard M. (1978). A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Communications of the ACM*, 120–126.
- Schemitt, A. G. (2025, Oktober 10). *Assessing the Impact of Post-Quantum Digital Signature Algorithms on Blockchains*. Retrieved from <https://arxiv.org/abs/2510.09271>
- Tovma, O. M., & Balagura, Dmytro Serhiiiovych. (2025). Comparative Analysis of the Performance of RSA, ECDSA, and CRYSTALS-Dilithium Digital Signature Algorithms in the Post-Quantum Era. *Computer and Information Systems and Technologies*, (pp. 112–113). Kharkiv, Ukraine.

AI Directory

System	Prompt	Usage
ChatGPT	“Create an abbreviation list for the following terms: ...”	Creation of the list of abbreviations; Content checked: partially revised
ChatGPT	“Suggest glossary entries for digital signature and post-quantum cryptography terms”	Drafts for glossary definitions; compared with specialist literature: partially revised
ChatGPT	“Improve the scientific language of this section: ...”	Wording aid for individual sections: partially revised
ChatGPT	“Check spelling and grammar: ...”	Spelling and grammar checker: unchanged
NotebookLM	“Create an outline for a thesis on the performance comparison of classical and post-quantum signature methods”	Structuring support: revised
Claude Code	“Check my Python script for comparing signature algorithms for errors and suggest appropriate corrections”	Revision of the Python script

Declaration of Originality

„Hiermit bestätige ich, dass ich die vorliegende Arbeit persönlich und selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder sinngemäß anderen Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Zeichnungen, Abbildungen und Tabellen in dieser Arbeit sind von mir selbst erstellt oder wurden mit einem entsprechenden Quellennachweis versehen. Diese Arbeit wurde weder in gleicher noch in ähnlicher Form von mir an anderen Hochschulen zur Erlangung eines akademischen Abschlusses eingereicht“.

A handwritten signature in black ink, appearing to be 'L. Wenzel'.

Betzdorf, 29.09.2026